

SelVeri Error Diagnostics

SelVeri
May 19, 2026

Contents

1 SelVeri Error Diagnostics	1
1.1 Diagnostic Shape	1
1.2 Error Hierarchy	2
1.3 Diagnostic Codes	3
1.4 Rendering Examples	4
1.5 Writing New Diagnostics	5
1.6 CLI Behavior	5

1 SelVeri Error Diagnostics

SelVeri reports user-facing diagnostics through `SelVeriError` subclasses instead of showing Python tracebacks during normal CLI execution.

The CLI catches `SelVeriError` and renders its attached `Diagnostic` with source locations, labels, notes, hints, fixes, and counterexamples when available. Unexpected Python exceptions are rendered as internal errors unless `--debug` is used.

1.1 Diagnostic Shape

The public diagnostic model lives in `src/selveri/diagnostics.py`.

Core fields:

```
Diagnostic(  
    code="SV-C001",  
    severity=DiagnosticSeverity.ERROR,  
    category="CompileError",  
    title="duplicate declaration",  
    message="variable `x` is already declared as `Int`",  
    span=span,  
)
```

Richer diagnostics can also attach:

- labels: primary and related source spans.

- notes: extra explanation, grouped by kind.
- hint: a concise repair suggestion.
- fixes: optional concrete fix messages and replacements.
- context: structured data for tools.
- counterexample: verification/runtime values that caused the error.

Example:

```
Diagnostic(
    code="SV-C001",
    severity=DiagnosticSeverity.ERROR,
    category="CompileError",
    title="duplicate declaration",
    message="variable `x` is already declared as `Int`",
    span=second_decl_span,
    labels=(
        DiagnosticLabel(second_decl_span, "declared again here", "primary"),
        DiagnosticLabel(first_decl_span, "original declaration of `x`", "secondary"),
    ),
    notes=(DiagnosticNote("previous declaration was here"),),
    hint="remove the second declaration or use a different variable name",
)
```

1.2 Error Hierarchy

The main error classes live in `src/selveri/errors.py`.

- `ParserError`
- `PreprocessorError`
- `CompilerError`
- `IRParseError`
- `IRRuntimeError`
- `VerifierRuntimeError`
- `VerificationError`
- `SpecDomainBoundsError`

All of them inherit from `SelVeriError` and carry a diagnostic attribute.

Simple legacy construction still works:

```
raise CompilerError("something went wrong", span=span)
```

Prefer factory functions for common cases:

```
raise duplicate_declaration_error(
    name="x",
    new_span=current_decl.span,
    original_span=previous_decl.span,
    original_type="Int",
)
```

1.3 Diagnostic Codes

Codes are stable user-facing identifiers. They are defined by `DiagnosticCode`.

Prefix	Area
SV-P000	Generic parser error
SV-Pxxx	Parser errors
SV-PP000	Generic preprocessor error
SV-PPxxx	Preprocessor errors
SV-C000	Generic compiler error
SV-Cxxx	Compiler errors
SV-R000	Generic runtime error
SV-Rxxx	Runtime errors
SV-IR000	Generic IR parse/runtime error
SV-IRxxx	IR parse/runtime errors
SV-V000	Generic verification error
SV-Vxxx	Verification errors
SV-Ixxx	Internal errors

Common specific codes:

Code	Meaning
SV-P001	Unexpected token
SV-P002	Missing block terminator
SV-PP001	Unclosed specification block
SV-PP002	Empty specification block
SV-PP003	Nested specification block
SV-C001	Duplicate declaration
SV-C002	Type mismatch
SV-C003	Unknown identifier
SV-C004	Invalid list index
SV-C005	Invalid return type
SV-R001	Division by zero
SV-R002	Index out of bounds
SV-R003	Invalid input value
SV-R004	Obtain failed
SV-V001	Assertion failed
SV-V002	Solver returned unknown
SV-V003	Temporal specification violated
SV-V004	Temporal specification not satisfied
SV-V005	Invalid START bound
SV-V006	Invalid END bound
SV-V007	Invalid specification

1.4 Rendering Examples

Duplicate declaration:

```
CompileError: duplicate declaration [SV-C001]
--> example.svi:2:1
  |
2 | x : Float;
  | ^^^^^^^^ declared again here
  |
note:
--> example.svi:1:1
  |
1 | x : Int;
  | ^^^^^^ original declaration of `x`

= variable `x` is already declared as `Int`
= note: previous declaration was here
= hint: remove the second declaration or use a different variable name
```

Missing block terminator:

```
ParseError: missing block terminator [SV-P002]
--> example.svi:1:1
  |
1 | if x < 3 then
  | ^^ `if` block starts here

= expected `fi` before end of input
= hint: close the conditional block with `fi`
```

Unclosed specification block:

```
PreprocessorError: unclosed specification block [SV-PP001]
--> example.svi:1:8
  |
1 | x := 1 { x > 0;
  |           ^ specification block starts here

= expected closing `}`
= hint: close the specification block with `}`
```

Verification counterexample:

```
VerificationError: assertion have failed [SV-V001]
--> example.svi:2:1
  |
2 | { x > 0 }
  | ^^^^^^^^ specification is not guaranteed

= the verifier ended up in a state where `x > 0` is false
```

1.5 Writing New Diagnostics

Use these rules when adding a new diagnostic:

1. Use a stable code from the right category.
2. Point span and the primary label at the most actionable location.
3. Add secondary labels for related declarations, definitions, or specifications.
4. Keep message short and factual.
5. Put repair guidance in hint.
6. Use Counterexample for values from Z3, runtime checks, and temporal traces.
7. Do not expose raw Python exceptions, raw Z3 model names, or internal IR details unless no source span exists.

For common errors, add a factory function in `src/selveri/errors.py`. Keep the renderer independent from compiler, parser, runtime, and verifier internals.

1.6 CLI Behavior

Normal mode:

```
selveri examples/diagnostics/fail/duplicate_declaration.svi
```

prints a formatted diagnostic and exits with status code 1.

Debug mode:

```
selveri examples/diagnostics/fail/duplicate_declaration.svi --debug
```

still renders `SelVeriError` diagnostics normally, but unexpected internal exceptions include the Python traceback.